
Reporte Técnico: SEGG-U2-P3G-5

Generación de Evidencias y Reporte de Resultados

Asignatura: Seguridad en el Desarrollo de Software

8 de julio de 2026

1 Objetivo

Generar un volumen representativo de registros de eventos utilizando la aplicación `VulnerableApp`, analizar la información registrada y elaborar un reporte técnico que documente los resultados obtenidos.

2 Descripción de las Pruebas Ejecutadas

Se utilizó un script automatizado en PowerShell (`GenerarTrafico_P3G_5.ps1`) para ejecutar los siguientes escenarios sobre la aplicación instrumentada:

- **Navegación General:** 30 peticiones a la página principal y acceso a todos los controladores para generar `CorrelationId`.
- **Búsquedas:** 100 búsquedas válidas, 20 vacías, 20 con caracteres especiales y 20 intentos de SQL Injection.
- **Autenticación:** 50 inicios de sesión exitosos, 100 intentos fallidos por contraseña y 20 intentos con usuarios inexistentes.
- **Comentarios:** 100 comentarios válidos, 30 con posibles cargas XSS y 20 comentarios vacíos.
- **API:** Consumo de endpoints (`/api/users`) 200 veces, incluyendo consultas a recursos inexistentes e identificadores inválidos.
- **Excepciones y Warnings:** Se provocaron excepciones no controladas mediante payloads SQL defectuosos y consultas intencionales para evaluar el *Exception Middleware* y el registro de advertencias.

3 Resultados Obtenidos y Respuestas al Análisis

Basado en el análisis de los archivos de log generados (`log-20260706.txt`) y las consultas estructuradas en el servidor de **Seq**, se extrajeron las siguientes métricas e identificaciones:

¿Cuántos eventos *Information* fueron registrados?

Se registraron un total de **2460** eventos de nivel *Information*.

¿Cuántos eventos *Warning* fueron registrados?

Se registraron **235** eventos de nivel *Warning* (incluyendo las advertencias por comentarios vacíos y el paso de parámetros inválidos).

¿Cuántos eventos Error fueron registrados?

Se registraron **376** eventos de nivel *Error*, provocados en su mayoría por excepciones no controladas e inyecciones SQL que fallaron de forma controlada en la base de datos.

¿Qué controlador generó más registros?

El `AuthController` generó la mayor cantidad de registros debido al alto volumen de intentos de autenticación planificados (aproximadamente **402** registros en total).

Cuál fue el endpoint con mayor número de solicitudes?

El endpoint `/Auth/Login` fue el más solicitado del sistema, acumulando más de **302** peticiones contabilizadas explícitamente en los logs.

¿Cuál fue la dirección IP con mayor actividad?

La dirección IP `::1` (localhost IPv6) registró la mayor actividad con **1142** coincidencias, dado que las pruebas automatizadas se ejecutaron localmente.

¿Cuántos intentos de autenticación fallidos existieron?

Existieron **120** intentos fallidos generados de manera directa en la última ventana de pruebas (100 con contraseña incorrecta y 20 con usuarios inexistentes). Sin embargo, el conteo total consolidado en el log histórico suma **295**.

¿Cuántos intentos de SQL Injection fueron identificados?

Se identificaron de forma inequívoca **30** intentos de inyección SQL explícitos generados por el script (20 mapeados en búsquedas y 10 diseñados para provocar excepciones en la persistencia), reflejados con múltiples alertas en Seq.

¿Cuántos posibles intentos de XSS fueron registrados?

Se registraron **30** posibles intentos de XSS (cargas maliciosas directas utilizando la sintaxis `<script>alert('XSS')</script>`) insertados de forma controlada mediante el controlador de comentarios.

¿Cuál fue la solicitud con mayor tiempo de ejecución?

Una petición de tipo `POST` dirigida a `/Auth/Login` registró un tiempo de ejecución máximo de **1220 ms**, derivado del coste computacional del cálculo de hashes de contraseña bajo el algoritmo `BCrypt`.

¿Fue posible localizar una petición específica utilizando únicamente el CorrelationId?

Sí. Gracias al middleware de `CorrelationId`, cada petición HTTP recibió un identificador único global (GUID) inyectado al `LogContext`. Esto permitió utilizar el filtro estructurado `CorrelationId = 'valor'` en Seq para trazar por completo la vida útil de una misma petición de principio a fin sin interferencias de otros hilos.

4 Evidencias de Código (Snippets)

4.1 Configuración de Serilog en Program.cs

A continuación se muestra el fragmento de configuración donde se inicializa la tubería de Serilog, habilitando sumideros para Consola, Archivo local con rotado diario, y la integración directa con la API de Seq:

```
1 using VulnerableApp.Data;
2
3 var builder = WebApplication.CreateBuilder(args);
4
5 // Configurar Serilog de forma centralizada
6 Log.Logger = new LoggerConfiguration()
7     .ReadFrom.Configuration(builder.Configuration)
8     .WriteTo.Console()
9     .WriteTo.File("Logs/log-.txt", rollingInterval: RollingInterval.Day)
10    .WriteTo.Seq("http://localhost:5341")
11    .Enrich.FromLogContext()
12    .Enrich.WithMachineName()
13    .CreateLogger();
14
15 builder.Host.UseSerilog();
16
17 // ... Continuación de la configuración de servicios
18
19 app.UseMiddleware<VulnerableApp.Middlewares.GlobalLoggingMiddleware>();
```

Listing 1: Configuración e instrumentación de Serilog en la canalización de ASP.NET Core.

4.2 Middleware de CorrelationId y Exception Middleware

La lógica de intercepción de peticiones HTTP, el cálculo de tiempos por telemetría con Stopwatch, la gestión de identificadores de correlación y la captura global de excepciones no controladas se consolidaron dentro de GlobalLoggingMiddleware.cs:

```
20 using Microsoft.Extensions.Logging;
21 using Serilog.Context;
22 using System;
23 using System.Diagnostics;
24 using System.Linq;
25 using System.Threading.Tasks;
26
27 namespace VulnerableApp.Middlewares
28 {
29     public class GlobalLoggingMiddleware
30     {
31         private readonly RequestDelegate _next;
32         private readonly ILogger<GlobalLoggingMiddleware> _logger;
33
34         public GlobalLoggingMiddleware(RequestDelegate next, ILogger<
35             GlobalLoggingMiddleware> logger)
36         {
37             _next = next;
38             _logger = logger;
39         }
40
41         public async Task InvokeAsync(HttpContext context)
```

```

41     {
42         // Lógica del Middleware de CorrelationId
43         var correlationId = context.Request.Headers["X-Correlation-Id"]
44             .FirstOrDefault() ?? Guid.NewGuid().ToString();
45         context.TraceIdentifier = correlationId;
46
47         var stopwatch = Stopwatch.StartNew();
48
49         using (LogContext.PushProperty("CorrelationId", correlationId))
50         {
51             try
52             {
53                 _logger.LogInformation("Iniciando petición HTTP {
54                     Method} {Path}", context.Request.Method, context.
55                     Request.Path);
56
57                 await _next(context);
58
59                 stopwatch.Stop();
60                 _logger.LogInformation("Petición HTTP {Method} {Path}
61                     completada en {ElapsedMilliseconds} ms con código {
62                     StatusCode}",
63                     context.Request.Method, context.Request.Path,
64                     stopwatch.ElapsedMilliseconds, context.Response.
65                     StatusCode);
66             }
67             catch (Exception ex)
68             {
69                 stopwatch.Stop();
70                 // Lógica del Exception Middleware ante fallos
71                 // catastróficos
72                 _logger.LogError(ex, "Excepción no controlada en HTTP
73                     {Method} {Path} después de {ElapsedMilliseconds} ms
74                     ",
75                     context.Request.Method, context.Request.Path,
76                     stopwatch.ElapsedMilliseconds);
77                 throw;
78             }
79         }
80     }
81 }

```

Listing 2: Implementación del Middleware Global de Logs, Correlación y Excepciones.

4.3 Ejemplo de un Controlador Instrumentado

Muestra de cómo los controladores aprovechan el contexto enriquecido y registran eventos con semántica limpia mediante parámetros estructurados en HomeController.cs:

```

1 public IActionResult Index(string search)
2 {
3     var sw = Stopwatch.StartNew();
4     var user = HttpContext.Session.GetString("User") ?? "Anónimo";
5     var ip = HttpContext.Connection.RemoteIpAddress?.ToString() ?? "
6     Desconocida";

```

```

7      _logger.LogInformation("Inicio HomeController.Index - Usuario: {User}
      IP: {IP} Parametros: search={Search}", user, ip, search);
8
9      try
10     {
11         if (string.IsNullOrEmpty(search))
12         {
13             sw.Stop();
14             _logger.LogInformation("Fin HomeController.Index -
              TiempoEjecucion: {ElapsedMilliseconds} ms", sw.
              ElapsedMilliseconds);
15             return View(new List<User>());
16         }
17         // ... Lógica adicional del negocio
18     }
19 }

```

Listing 3: Instrumentación contextual dentro del controlador HomeController.

5 Conclusiones

La implementación de un mecanismo centralizado de observabilidad utilizando **Serilog** y el servidor de ingestión estructurada **Seq** permite monitorear eficientemente el estado integral de la aplicación ante ráfagas de peticiones legítimas o maliciosas.

La inyección estructurada del `CorrelationId` demuestra ser una técnica vital e imprescindible para el aislamiento y rastreo completo de flujos complejos o erróneos sumergidos en un mar de miles de eventos concurrentes. Por otro lado, la disposición estratégica del *Exception Middleware* garantiza el principio de contención frente a fallas no controladas, asegurando el almacenamiento persistente del stack trace completo, lo cual optimiza drásticamente las tareas críticas de respuesta ante incidentes y el posterior análisis forense corporativo de la aplicación.